

串連系統 v2.0

管理層決策輔助系統 · 演算法與資料邏輯文件

本文件說明系統使用的核心演算法、評分公式、資料結構與決策邏輯。適合教師檢閱、新進工程師交接、論文撰寫附錄。

專案名稱	串連系統 v2.0 (Chuanlien System)
應用場景	20 – 50 人規模投資公司的管理層決策輔助
前端技術	React 19 + TypeScript + Vite 6 + Tailwind CSS v4 + Framer Motion + Recharts
後端服務	Firebase Authentication + Cloud Firestore
核心演算法	BM25F、TF-IDF、Gini Coefficient、Empirical Percentile、Weighted Load Model
作者	資管導論 第 13 組
文件版本	v2.0 · 2026-05

目錄

1. 系統總覽與核心模組
2. 加權員工負載模型 (Weighted Load Model)
3. 卡點分位數風險分析 (Empirical Percentile)
4. BM25F 歷史案件搜尋引擎
5. 智能案件推薦 (Smart Recommendation)
6. 部門互動網絡分析
7. 組織風險指數 ORI (Organizational Risk Index)
8. 組織健康度 6 維雷達
9. 資料同步機制與保護邏輯
10. 結語：方法論選擇背後的设计取舍

1. 系統總覽與核心模組

串連系統 v2.0 是面向中小型投資公司管理層的決策輔助平台，整合週報、案件交接、決策追蹤、員工負載分析、卡點風險預警、歷史案件檢索、組織網絡視覺化、會議準備自動化等 10 個模組。系統的設計核心並非「資料庫」，而是「演算法層」——在原始資料之上，用統計、資訊檢索、圖論方法把瑣碎事件融合成「組織儀表板」。

1.1 模組與對應演算法

週報填寫	Markdown 格式 + 關鍵字 tokenize
案件交接	簽收狀態機 + 逾時 SLA 監控
決策追蹤	三狀態（執行中/逾期/已完成）+ 平均完成天數
員工負載	加權負載模型 + Gini 係數 + Percentile Ranking
歷史搜尋	BM25F + 多 n-gram + 同義詞 + Substring Boost
卡點分析	Empirical Percentile（同類別 / 全公司歷史對照）
組織分析	雙向 Adjacency Matrix + 力導向圖 + 單向溝通偵測
會議準備	事件聚合 + 風險排序
組織健康	6 維加權雷達 + 12 週時間序列 + Local Minima 拐點
智能推薦	BM25F + 同類別過濾 + 解決天數中位數

2. 加權員工負載模型

單純計算「每人處理幾件案件」會嚴重低估近期高強度工作，也忽略案件複雜度。本模組採用加權負載分數（Weighted Load Score），融合四個訊號：

2.1 計算公式

$$\text{loadScore} = \text{timeWeightedCases} \times 1.5 + \text{blockerLoad} \times 2.0 + \text{mentionsWeighted} \times 0.8 + \text{handoffLoad} \times 1.0$$

2.2 各分量說明

timeWeightedCases	員工被指派的案件數，依「離本週多遠」做指數衰減——本週的案件權重 1.0，1 週前 0.7，2 週前 0.5……故近期高強度工作會被放大。
blockerLoad	員工目前負責的活躍卡點數 $\times$ 2.0 加權，因為卡點代表「正在燒」的工作，比一般案件更耗注意力。
mentionsWeighted	員工在他人週報中被提及的次數（共同負責、跨部門協作）。也帶時間衰減。權重 0.8 是為了避免「常被提到但實際沒分配工作」者被高估。
handoffLoad	員工作為交接接收者的待簽收件數，含逾時加成。

2.3 過載判定（Percentile-based）

計算所有員工的 loadScore 分布後，以下列門檻判定等級：

overload	loadScore $\geq$ 25 或 percentile $\geq$ 90
high	loadScore $\geq$ 15 或 percentile $\geq$ 75
normal	loadScore $\geq$ 6
low	loadScore $\geq$ 1
idle	loadScore $<$ 1（完全沒被分配工作的員工，作為流動人力儲備）

為什麼用 Percentile 而不是固定門檻？

公司規模不同、產業淡旺季不同，「絕對的 25 分」對小公司是過載、對大公司是普通。Percentile 自動適應分布，無論公司有 10 人還是 100 人都能正確找出「相對最忙的」。

3. 卡點分位數風險分析

傳統卡點告警常用「卡了 N 天」這種絕對天數，但不同類別的合理時長差異極大——法遵類平均 7-8 天合理、決策類平均 8-10 天合理、跨部門類平均 4-5 天就該升級。本系統用 Empirical Percentile（經驗分位數），把當前卡點的天數，對照過去同類別歷史的解決天數分布，計算「超過了多少 %」。

3.1 計算步驟

1. 從 SEED_HISTORY（53 筆歷史案件）依 tags 過濾出同類別案例。
2. 提取每筆案例的解決天數（從 outcome 字串“已解決 · N 天” parse）。
3. 若同類樣本 ≥ 5 筆 → 用同類分布；否則 fallback 用全公司歷史。
4. 計算 P75 / P90 / P95，以及當前卡點所處的 percentile rank。

3.2 風險等級判定

critical（極高風險）	$\text{currentDays} \geq \text{P95} \rightarrow$ 已超過 95% 同類歷史，立刻召開協調會議
high（高風險）	$\text{currentDays} \geq \text{P90} \rightarrow$ 建議本週內升級處理
medium（關注中）	$\text{currentDays} \geq \text{P75} \rightarrow$ 進入關注區，追蹤後續進度
normal（正常）	$\text{currentDays} < \text{P75} \rightarrow$ 仍在正常處理時程內

3.3 處理樣本不足的情境

若同類歷史案例不足 5 筆（無統計顯著性），系統會：

- (a) 改用全公司歷史案例池，至少有 53 筆樣本可用；
- (b) 標記 `hasData = false`，提示使用者「無同類歷史可比對」；
- (c) 退化為時間 SLA 提醒（21 天以上轉 high、14 天以上轉 medium）。

公式：percentile rank

$$\text{percentile} = (\text{count of historical_days} \leq \text{currentDays}) / \text{total_samples} \times 100$$

4. BM25F 歷史案件搜尋引擎

早期版本用 TF-IDF + Cosine Similarity，但有幾個缺點：(1) TF 採線性計分，「出現 5 次」和「出現 50 次」差距過大；(2) 字元 bigram 切「東京中央銀行」會被拆碎；(3) 標題權重等同內文，搜尋公司名常被誤判。v2.0 升級為 BM25F (Okapi BM25 with Field weighting)，這是 Elasticsearch 與 Lucene 的核心演算法。

4.1 BM25 TF 飽和函數

```
tf_normalized = (tf × (k1 + 1)) / (tf + k1 × (1 - b + b × (len / avgLen)))
```

其中 $k1 = 1.5$ (TF 飽和速度)、 $b = 0.75$ (長度正規化強度)。此函數會讓 TF 在多次出現後趨於飽和，避免長文章天然吃虧。

4.2 欄位權重 (Field Weights)

title (標題)	權重 5.0 — 公司名 / 案件主題出現於標題，相關性遠高於內文
tags (標籤)	權重 4.0 — 卡點類別、行業別、速度標籤
summary (摘要)	權重 2.0 — 一句話描述
outcome (結論)	權重 1.5 — 「已解決 · N 天」
owner (負責人)	權重 1.0
detail (詳細)	權重 1.0 — 背景 / 過程 / 估值 / 結果 / 經驗

4.3 中文 Tokenization：多 n-gram 混合

中文沒有明確詞界，本系統同時切 1-gram、2-gram、3-gram：

「東京中央銀行」→ 東 / 京 / 中 / 央 / 銀 / 行 / 東京 / 京中 / 中央 / 央銀 / 銀行 / 東京中 / 京中央 / 中央銀 / 央銀行

3-gram 特別重要，因為投資領域多為 3-4 字專有名詞（投委會、董事會、伊勢島飯店）。

4.4 Substring Boost：解決 n-gram 拆碎問題

若使用者輸入「東京中央銀行」，光靠 n-gram 命中分數會被「銀行」這種 high-DF 的字稀釋。因此另加一條規則：

```
if document.contains(query_string) → score × 1.8
if title.contains(query_string) → score × 1.4
```

4.5 同義詞表 (Synonym Normalization)

金融 / 投資領域常見同義詞，在 tokenize 前一律正規化為 canonical form：

- 募資 ≈ 融資 ≈ 募款 ≈ fundraising
- 盡調 ≈ 盡職調查 ≈ due diligence ≈ DD

- NDA ≈ 保密協議 ≈ 保密
- LOI ≈ 意向書
- 投委會 ≈ 投資委員會 ≈ IC
- 估值 ≈ valuation ≈ 定價
- 退場 ≈ exit ≈ 出場
- 法遵 ≈ compliance ≈ 合規
- 風控 ≈ 風險管理 ≈ risk
- Pre-A ≈ PreA ≈ pre-a ≈ 種子輪後

4.6 IDF：Robertson-Sparck-Jones 公式

$$\text{idf}(t) = \log(1 + (N - \text{df}(t) + 0.5) / (\text{df}(t) + 0.5))$$

N = 文件總數、df(t) = 含詞 t 的文件數。+0.5 是 BM25 標準的 Lidstone smoothing，避免極端值。此公式對罕見詞（df 小）給高分，對「的、了、是」等 stop word 給接近 0 分。

5. 智能案件推薦

卡點 / 交接的詳情頁中，系統會自動跑一次 BM25F 搜尋，找出過去 3 筆最相似的歷史案。這把「歷史搜尋」從被動工具轉成主動決策輔助。

5.1 推薦觸發時機

卡點分析頁	點開卡點 → 推薦類似類別 + 公司的歷史案
交接 Modal	點開交接單 → 推薦類似案件背景的歷史案
使用 query	標題 + 描述 + 類別 + 案件 ID (caseId) 串接後送入 BM25F
過濾條件	relevance > 20%、取前 3 筆
附加資訊	顯示相關度 %、解決天數、速度標籤（快/正常/較慢/延誤）、平均解決天數

為什麼不用 LLM Embedding？

投資公司資料涉及機密，使用 OpenAI / Anthropic Embedding API 有資料外洩疑慮。BM25F 完全在前端跑、零外部 API、可解釋（為什麼推薦這筆？因為標題命中『東京中央銀行』）。對只有 50-200 筆歷史案的中小型企業，BM25F 的準確度足以匹敵 Embedding。

6. 部門互動網絡分析

從週報文字與交接紀錄中萃取「誰提到誰」的訊號，建構部門間的有向加權圖（Directed Weighted Graph）。

6.1 邊權重來源

週報 mention	部門 A 的週報文字中提到部門 B 的次數（用正則匹配部門名/簡稱）
交接量	部門 A 發出給部門 B 的交接單數量
加總	兩者加總，得到 $A \rightarrow B$ 的邊權重 $\text{weight}(A, B)$

6.2 視覺化

使用力導向圖（Force-directed Layout），節點為部門，邊粗細代表協作密度。節點大小依「對外溝通總量」決定。系統提供時間範圍切換（4 週 / 8 週 / 3 月 / 半年 / 1 年 / 全部），點任一節點可看該部門相關的週報與交接清單。

6.3 單向溝通偵測（Asymmetric Communication）

組織內最常被忽視的病徵是「A 部門一直找 B，但 B 從不回應」。系統會自動掃描所有部門對 (A, B)，當 $\text{weight}(A, B) \geq 5$ 且 $\text{weight}(B, A) = 0$ 時，標記為單向溝通，計入 ORI 的 CDC 維度。

7. 組織風險指數 ORI

ORI (Organizational Risk Index) 是融合「人力集中、決策延遲、卡點尾端風險、跨部門溝通」四個維度的綜合風險指標。0 = 最健康，200 = 最高風險。

7.1 公式

$$\text{ORI} = 0.35 \times \text{HCC} + 0.25 \times \text{DL} + 0.25 \times \text{BT} + 0.15 \times \text{CDC}$$

7.2 四個驅動因子

HCC (Human Capital Concentration)	人力集中度。100 + (Gini - 0.35) × 400 + (top1占比 - 20%) × 200 + 離群值數 × 8。用 Gini 係數衡量負載分布不公平，加上 top1 員工占整體負載比例。
DL (Decision Latency)	決策延遲。100 + (avgCompletionDays - 14) × 4 + 逾期數 × 12。14 天為基準線，每超過 1 天扣 4 分，每件逾期重罰 12 分。
BT (Blocker Tail Risk)	卡點尾端風險。100 + (avgPercentile - 50) × 1.5 + P90數 × 8 + P95數 × 12。重點懲罰「進入歷史 90% 以上」的長尾卡點。
CDC (Cross-Dept Comm Health)	跨部門溝通。100 + 單向組數 × 18 + 單向總量 × 0.5。識別組織內的「溝通斷層」。

7.3 五級告警

$\text{ORI} \geq 175$	今天要花時間 — 下面這幾件事不解掉，整週都會被拖住
$\text{ORI} \geq 150$	要注意 — 有幾件事在惡化，建議今天看一下
$\text{ORI} \geq 125$	可關注 — 整體穩定，但有少量需注意事項
$\text{ORI} \geq 100$	還可以 — 整體穩定，無重大警示
$\text{ORI} < 100$	順利 — 本週公司運作正常

8. 組織健康度 6 維雷達

ORI 用 0 – 200 反向計分（越低越好）對管理層較難直覺理解。v2.0 新增「組織健康度」儀表板，採 0 – 100 正向計分（越高越好），並擴展為 6 個維度的雷達圖，搭配 12 週時間序列追蹤。

8.1 整體分數加權

```
overall = blockerHealth × 0.22 + decisionTimeliness × 0.18  
+ handoffSmoothness × 0.15 + loadBalance × 0.18  
+ crossDept × 0.12 + reportQuality × 0.15
```

8.2 六個維度詳解

卡點健康 (22%)	$100 - P95\text{數} \times 15 - P90\text{數} \times 7 - \max(0, \text{avgPercentile} - 50) \times 0.8$
決策及時 (18%)	$100 - \max(0, \text{avgCompletionDays} - 14) \times 3 - \text{逾期數} \times 10$
交接流暢 (15%)	$50 + \text{完成率} \times 50 - \text{逾時24h+ 件數} \times 8$
負載均衡 (18%)	$100 - \max(0, \text{Gini} - 0.35) \times 200 - \text{過載員工} \times 8$
部門協作 (12%)	$100 - \text{單向溝通組數} \times 15$
週報品質 (15%)	$\text{繳交率} \times 60 + \text{內容字數} \times 0.3 + \text{卡點欄填寫率} \times 10$

8.3 12 週時間序列與拐點偵測

為了讓管理者觀察組織健康的「軌跡」，系統會回溯計算過去 12 週的整體分數。對於時間序列上的低谷點，採 Local Minima Detection：

```
if series[i] < series[i-1] - 3 AND series[i] < series[i+1] - 3 → ■■■■■■
```

拐點在折線圖上用紅點標記。使用者點選任一週後，下方會 pin 住該週的事件清單（如「3 件極高風險卡點 → 卡點分析」「2 筆決策逾期 → 決策追蹤」），且每個事件 chip 可點擊跳轉至對應頁面 — 將分析發現直接落地為行動。

為什麼建兩個指標（ORI + 健康度）？

ORI 沿用至內部 dashboard 與 sidebar 角標，提供熟悉 0 – 200 風險刻度的使用者。健康度是面向「教授 / 簡報觀眾」的直觀視覺化，0 – 100 正向計分易於 storytelling。兩者底層共享同一套原始資料（loads, decisions, blockers, network），不重複計算。

9. 資料同步機制與保護邏輯

前端 React state 與 Firestore Cloud 之間採用 樂觀同步 (Optimistic Sync)：

9.1 寫入流程

1. 使用者操作（新增、修改、刪除）立即更新 React state（畫面瞬間反應）
2. useEffect 監聽 state 變化，觸發 saveDocumentCollection()
3. 寫入 Firestore，更新 syncStatus (idle / syncing / error)
4. Sidebar 即時顯示同步狀態指示燈

9.2 讀取與 SEED 保護

系統啟動時會從 Firestore 拉取資料，但有「保護條款」避免覆蓋使用者既有資料：

讀到資料量 ≥ 10	尊重 Firestore 版本，不覆蓋
讀到資料量 < 10	視為「首次使用 / 空資料庫」，用 SEED_REPORTS / SEED_HANDOFFS / SEED_HISTORY 填充
讀到舊版格式	如週次標籤含「第 N 週」舊格式 → 重置為新 SEED
使用者新增 / 修改	立即寫回 Firestore，永久保留

9.3 認證與權限

採 Firebase Authentication 的 Email/Password 登入。使用者角色 (admin / manager / member) 由 inferUserProfile() 依 email 推斷，並與 Firestore users collection 合併。不同角色看到不同 sidebar 項目 (admin 看全部、manager 看大部分、member 看週報與交接)。

10. 結語：方法論選擇背後的设计取舍

本系统的核心设计理念是「用恰当的演算法解决恰当规模的问题」。我们不采用 LLM、Transformer 这类重型工具，原因是：

- 1. 资料量小：53 笔历史案、240 笔週报、150 笔交接，BM25F 比 Embedding 更可靠且零成本。
- 2. 可解释性：管理层需要知道「为什么推荐这笔」，BM25F 可逐项列出命中词贡献；Embedding 不行。
- 3. 机密敏感：投资公司资料不能送 cloud API，本系统全部在前端 / 自有 Firebase 跑。
- 4. 计算成本：BM25F、Gini、percentile 都是 $O(N)$ 或 $O(N \log N)$ ，在浏览器毫秒级完成。

10.1 演算法选用对照

資訊檢索	BM25F (Lucene 同款) — 而非 TF-IDF / cosine 或 sentence embedding
風險量化	Empirical Percentile — 而非絕對天數門檻
不平均度	Gini 係數 (經濟學標準) — 而非標準差
時間衰減	Exponential Decay — 而非滑動視窗
拐點偵測	Local Minima — 而非機器學習 anomaly detection
組織網絡	Force-directed Graph + 雙向 Matrix — 而非單純 list

10.2 後續可擴充方向

- 多元迴歸預測卡點解決時間 (純 JS 可實作，無需外部模型)
- What-if 模擬器 (拉滑桿模擬「增加 1 名法遵專員」對卡點數的影響)
- 員工技能向量化 (從週報內容萃取個人專長標籤)
- 董事會 PDF 一鍵匯出 (自動產出本週摘要簡報)
- 語音輸入週報 (Web Speech API)

「資料越少，演算法的選擇越重要。」
—— 串連系統設計哲學